UML e casi d’uso

Cos’è un Modello

Un modello è un’**astrazione** di un sistema (o del dominio di interesse) usata per descriverne la **struttura** e/o il **comportamento**.

Caratteristiche principali:

- **Cattura gli aspetti importanti** del sistema, tralasciando i dettagli irrilevanti.
- **Espresso tramite formalismo**, rendendone l’uso e la comprensione più semplice.

Diverse Viste su un Modello

- Un sistema non viene descritto da una singola prospettiva.
- È definito tramite un **insieme di viste** che, nel loro insieme, offrono una rappresentazione completa del modello.

Ruolo dei Modelli

I modelli sono strumenti fondamentali per:

- **Documentazione** del sistema.
- **Comunicazione** tra membri del team di sviluppo.
- **Discussione e collaborazione** nella progettazione.

Sono al centro dei moderni processi di sviluppo software.

Come si Modella un Sistema

- Si sceglie se descrivere un modello:
  - **Statico** (struttura, componenti, relazioni).
  - **Dinamico** (comportamento, sequenze di azioni).

- Si definisce il **livello di astrazione**: alto livello (concettuale) o basso livello (dettagli implementativi).

Come si Rappresenta un Modello

- Con un **linguaggio formale** (es. UML con semantica rigorosa).
- Con un **linguaggio semi-formale** (es. diagrammi, schemi grafici).

Come si Usa un Modello

- **Bozza (sketch)**: modello incompleto, usato per descrizioni iniziali o per esplorare alternative progettuali.
- **Progetto dettagliato (blueprint)**: contiene tutti i dettagli necessari per sviluppare un sistema pronto all'esecuzione.
- **Modello eseguibile**: così completo e preciso da consentire la generazione automatica del codice.

UML – Unified Modeling Language

- Linguaggio di modellazione unificato.
- Supporta la **descrizione e progettazione di sistemi software**, in particolare **object-oriented**, ma applicabile anche ad altri paradigmi.
- **Famiglia di notazioni grafiche** per rappresentare sistemi da diversi punti di vista.
- I diversi modelli sono correlati e descritti tramite un **meta-modello comune**.
- Generalmente **facile da comprendere** anche da clienti e utenti.

Applicazioni di UML

- Descrizione **strutturale (statica)** vs descrizione **comportamentale (dinamica)**.
- Bozza di progetto vs progetto finale.
- Descrizione del **dominio** vs descrizione della **struttura del codice**.

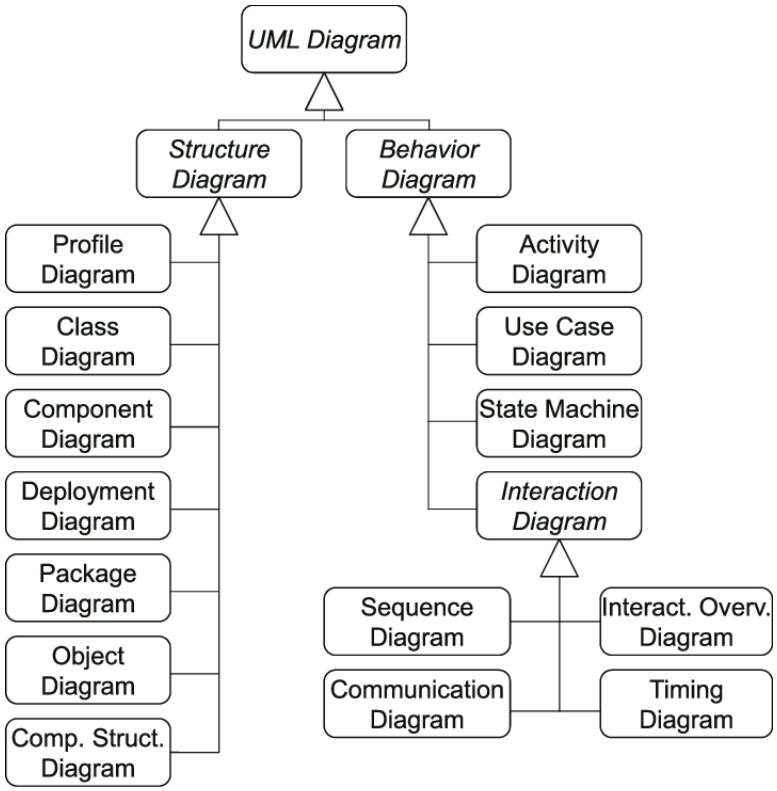
Storia di UML

- **<1994**: esistevano molti linguaggi e metodi OO.
- **1994**: primo tentativo di unificazione con il linguaggio **Fusion**.
- **1994**: Booch e Rumbaugh propongono UML nella **Rational Corporation**.
- **1996**: OMG lancia una prima proposta di standardizzazione.
- **1997**: approvato lo standard UML 1.0.
- **2000**: UML 1.4.
- **2006**: UML 2.0 e introduzione di **Model-Driven Architecture (MDA)**.
- **Dal 2017**: stabile alla versione **2.5.1**.

Modello Statico vs Modello Dinamico

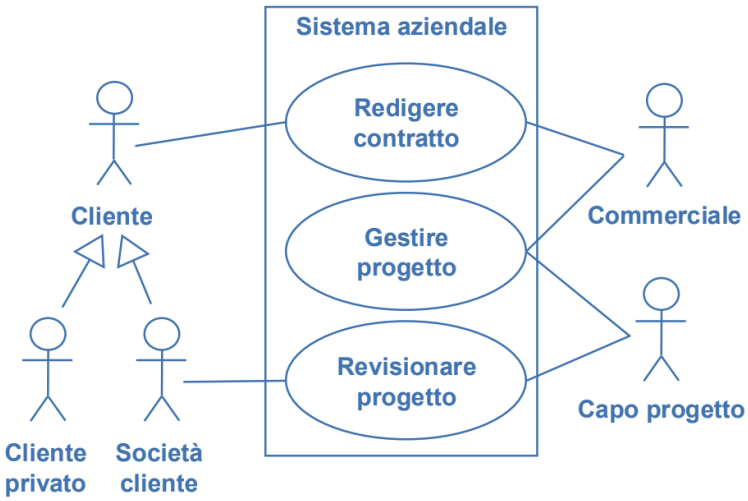
Tipo di modello	Descrizione
Statico	Rappresenta entità e relazioni senza considerare il tempo. Include concetti del dominio, componenti architetturali, classi di realizzazione.
Dinamico	Modella il comportamento delle entità definite nel modello statico. Si concentra su sequenze di eventi, interazioni e stati nel tempo.

Diagrammi in UML



Primo Diagramma: Casi d’Uso

- Serve a **modellare i requisiti funzionali** di un sistema software.
- Aiuta a:
  1. Raccogliere i requisiti funzionali.
  2. Elaborarli.
  3. Documentarli.



Relazione con i Modelli UML

Tipo di modello	Strumento usato
Statico	Diagramma dei casi d'uso
Dinamico	Narrative associate ai casi d'uso (scenari dettagliati)

Costruzione del Diagramma dei Casi d'Uso

Per modellare i requisiti funzionali con un diagramma dei casi d'uso, si seguono questi passi:

1. **Individuare il confine del sistema**
   - Determina quali elementi appartengono al sistema e quali sono esterni.
2. **Individuare gli attori**
   - Chi interagisce con il sistema (utenti, altri sistemi, tempo).
3. **Individuare i casi d'uso**
   - Funzionalità o servizi che il sistema deve offrire.
4. **Individuare le relazioni attore-caso d'uso**
   - Chi utilizza quale funzionalità e come.
5. **Specificare il caso d'uso**

- Descrizione testuale (narrativa) dettagliata di ciascun caso d’uso.

Notazione del Diagramma dei Casi d’Uso

Elemento	Descrizione	Note
Attore	Rappresenta chi interagisce con il sistema	Nome in maiuscolo; è una classe
Caso d’Uso	Rappresenta un compito o servizio offerto dal sistema	Nome in maiuscolo; verbo che denota il compito
Relazione	Connessione significativa tra attori e casi d’uso	Rappresenta interazioni e sequenze di messaggi
Confine del Sistema	Rettangolo che circonda i casi d’uso	Indica cosa appartiene al sistema oggetto del modello

NARRATIVA DI UN CASO D’USO

È un documento che descrive il **modello dinamico** di un sistema, cioè come un caso d’uso si svolge nel tempo, dal punto di vista degli attori, incluso chi lo avvia (attore primario).

Caratteristiche principali

- Indica **inizio, passi intermedi e fine** del caso d’uso.
- Include **condizioni eccezionali**.
- Specifica **precondizioni** e **postcondizioni**.

Struttura del documento

Campo	Descrizione
Nome	Nome del caso d’uso
ID	Identificatore univoco
Breve descrizione	Riassunto di 1-2 righe del caso d’uso
Attore primario	Chi avvia il caso d’uso
Attori secondari	Altri attori che interagiscono con il caso d’uso

Campo	Descrizione
Precondizioni	Condizioni che devono essere vere prima dell'esecuzione del caso d'uso
Sequenza principale degli eventi	Sequenza ordinata di passi che descrive lo svolgimento del caso d'uso
Postcondizioni	Condizioni che devono essere vere al termine della sequenza principale
Sequenze alternative degli eventi	Ramificazioni, errori o interruzioni della sequenza principale

Scenario

- Uno scenario è una **istanza specifica di un caso d'uso**.
- Descrive una sequenza di passi che produce un **risultato osservabile dagli attori**.
- La sequenza principale dello scenario porta dalle **precondizioni** alle **postcondizioni**.



Precondizioni e Postcondizioni

Precondizione: è un **asserzione logica** che deve essere vera **prima** dell'inizio del caso d'uso.

- Non descrive cosa fa l'attore o il sistema, ma lo **stato necessario** perché il caso d'uso possa partire.
- Esempio: "L'utente è registrato nel sistema" o "Il carrello non è vuoto".

Postcondizione: è un **asserzione logica** che deve essere vera **dopo** la conclusione del caso d'uso.

- Descrive il **nuovo stato del sistema o dell'utente** al termine della sequenza principale.
- Esempio: "L'ordine è stato salvato nel database" o "Il profilo utente è aggiornato".

Caratteristiche comuni:

- Si esprimono come predicati o formule logiche.
- Valgono **vero o falso** in uno stato specifico.
- Non sono mai azioni o passi operativi.
- Sono sempre **necessariamente vere** nello stato che rappresentano (pre o post esecuzione).

Nome	Registra account
ID	CU143
Breve descrizione	Un utente si registra al sistema
Attore primario	Utente
Attori secondari	Nessuno
Precondizioni	L'utente ha cliccato sul link *Registrati* nella pagina principale del sito web
Sequenza degli eventi principale	1. Il sistema mostra il modulo di registrazione con i campi nome, cognome, email, password e conferma password 2. ….
…	



SBAGLIATO! Quanto scritto nella precondizione è l'azione con cui l'utente inizia il caso d'uso

Nome	Registra account
ID	CU143
Breve descrizione	Un utente si registra al sistema
Attore primario	Utente
Attori secondari	Nessuno
Precondizioni	**L'utente vuole registrarsi**
Sequenza degli eventi principale	**1. L'utente clicca sul link *Registrati* nella pagina principale del sito web** 2. Il sistema mostra il modulo di registrazione con i campi nome, cognome, email, password e conferma password 3. ….
…	

Flusso di una narrativa di caso d'uso

Parallelo con le triple di Hoare:

$$\{Precondizione\}\ SequenzaPrincipale\ \{Postcondizione\}$$

- Ogni stato σ che soddisfa la precondizione:
  - L'esecuzione della SPE produce uno stato $\sigma\prime$
  - Lo stato $\sigma\prime$ soddisfa la postcondizione.
- **Eccezioni** o sequenze alternative:
  - In caso di imprevisti descritti nella sequenza alternativa, la postcondizione principale **non è garantita**.

Sequenza principale degli eventi (SPE)

- Lista numerata di passi.
- Struttura di ogni passo:

```
<numero>. <soggetto> <azione> <complementi>
```

- Il primo passo (attivazione) **è sempre compiuto dall'attore principale**.

- Possibili costrutti di controllo:

1. **Iterazione**

```
2. For (condizione)
2.1 ...
2.2 ...
```

2. **Condizione**

```
3. Se (condizione)
3.1 ...
3.2 ...
Altrimenti // opzionale
3.3 ...
```

3. **Ciclo While**

```
4. While (condizione)
4.1 ...
4.2 ...
```

Inclusione di un caso d'uso

Definizione:

Un caso d'uso **1** incorpora l'interazione descritta da un caso d'uso **2**, come parte della sequenza di passi del caso d'uso 1.

Caratteristiche principali:

- La relazione `<<include>>` punta al caso d'uso che viene incluso (caso 2).
- Funziona come una **chiamata di funzione**: il caso 1 "invoca" il caso 2.
- Serve a **riutilizzare comportamenti comuni** tra più casi d'uso.

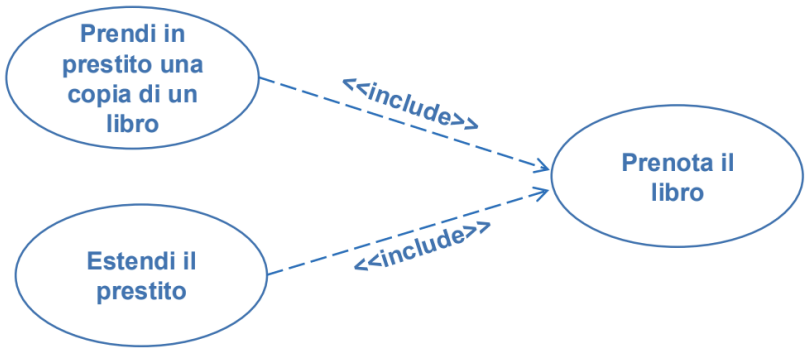
- È rappresentata graficamente come una **freccia tratteggiata** dal caso 1 verso il caso 2, annotata con `<<include>>`.

Stereotipi in UML:

- Gli **stereotipi** sono parole chiave tra `<< >>` che annotano elementi di un diagramma.
- Specificano il ruolo o la natura dell'elemento annotato.
- Nel caso dell'inclusione, `<<include>>` indica che un caso d'uso dipende dall'altro.

Narrativa:

- La **narrazione del caso 1** deve menzionare l'esecuzione del caso incluso (caso 2) nel punto opportuno della sequenza principale degli eventi (SPE).
- L'inclusione non modifica il flusso principale, ma integra passi comuni.



Estensione di un caso d'uso

Definizione:

Un caso d'uso **1** può includere un'interazione descritta da un caso d'uso **2**, ma **1 non dipende da 2**. L'estensione viene attivata solo in **alcune condizioni particolari**, non sempre.

Caratteristiche principali:

- La relazione `<<extend>>` punta al caso d'uso che viene esteso (caso 1 viene esteso da caso 2).
- Funziona come un **blocco opzionale o condizionale**: il caso 2 viene eseguito solo se si verifica la condizione di estensione.

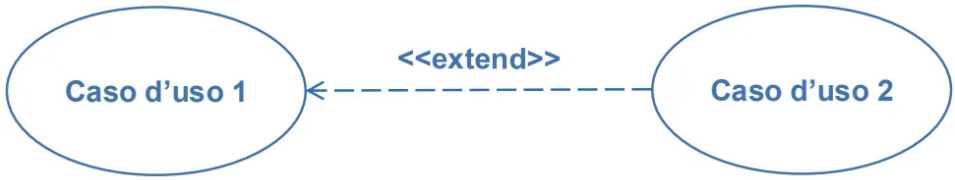
- È utile per **gestire scenari alternativi o comportamenti opzionali**.
- Rappresentata graficamente come una **freccia tratteggiata** dal caso d'uso esteso verso il caso d'uso opzionale, annotata con `<<extend>>`.

Stereotipi in UML:

- Gli **stereotipi** sono parole chiave tra `<< >>` che annotano elementi di un diagramma.
- Nel caso dell'estensione, `<<extend>>` indica che il caso d'uso opzionale si aggiunge al caso esteso solo in certe condizioni.

Narrativa:

- La **narrazione del caso 1** può menzionare l'esecuzione del caso esteso (caso 2) **solo se la condizione di estensione è soddisfatta**.
- Permette di **mantenere separati i comportamenti opzionali** dal flusso principale del caso 1, migliorando la leggibilità e il riuso.



Extension Points e Condizioni

Concetto:

- Le estensioni (`<<extend>>`) sono **opzionali** dal punto di vista del caso d'uso base.
- Non vengono eseguite sempre, ma solo quando si verifica una **condizione specifica**.

Extension Points:

- Servono a identificare **il punto preciso** nel flusso del caso d'uso base in cui l'estensione può essere applicata.
- Ogni extension point può avere un **nome** e indicare **quando** il caso d'uso esteso viene invocato.

Condizioni:

- La freccia `<<extend>>` può essere annotata con un **vincolo** o una **condizione logica**.
- La condizione stabilisce se l'estensione deve essere eseguita o meno.
- Sintassi tipica:

  ```
  <<extend>> [condizione]
  ```

  Questo permette di modellare **comportamenti opzionali** senza complicare la sequenza principale del caso d'uso base.